

Project Structure Guide

ParSU e-Feedback — Integrated Student Complaint and Feedback Management System with Institutional Analytics

This document walks through every folder in the ParSU e-Feedback codebase, from the project root down through its subfolders, and explains what each one is for and what it contains. It is written to be read without opening the code: folder names are given in the order they appear on disk, and each entry says what role it plays in the system.

How the project is organised

The codebase follows a feature-based layout on top of the Next.js App Router. Four layers matter most, and knowing them makes the rest of this document easy to follow:

src/app/ The routes. Each folder is a URL segment; `page.tsx` is a screen and `route.ts` is an API endpoint. This layer only gathers data and renders — it delegates every decision downwards.

src/features/ The business logic, grouped by domain (auth, complaints, routing, SLA, notifications, reports). Validation schemas and services live here, so a rule is written once and used by every screen.

src/models/ The database schema — one Mongoose model per MongoDB collection, describing shape and relationships only.

src/lib/ Shared infrastructure: the database connection, environment validation, authentication config, storage, email, logging and rate limiting.

Two supporting layers sit alongside them: `src/components/` holds the reusable interface pieces the screens are built from, and `src/proxy.ts` with `src/middleware/` gate every request by role before a route is even reached. Documentation lives in `docs/`, email templates in `emails/`, and the browser-driven tests in `tests/e2e/`.

Generated 2026-09-03. Folders are shown in blue and files in black; indentation shows nesting. Generated folders (`node_modules`, `.next`) and the build cache are listed for completeness but contain no source.

1. Project Root

Configuration, tooling and top-level folders. Everything here configures how the app is built, linted, tested and deployed — no application logic lives at this level.

README.md Project overview: what the system is, the stack (Next.js 16, MongoDB, R2, Resend, Upstash), setup steps and the npm scripts.

package.json Dependency manifest and scripts (dev, build, start, lint, typecheck, format, test, test:e2e).

package-lock.json Exact locked dependency tree — guarantees identical installs across machines.

next.config.ts Next.js config. Its one real setting marks pdfkit as a server-external package so its font-metric files resolve at runtime and PDF report generation does not fail.

tsconfig.json TypeScript compiler options (strict mode, the `@/*` path alias pointing at `src/`).

tsconfig.tsbuildinfo Incremental-build cache written by tsc. Generated, not source.

next-env.d.ts Auto-generated Next.js type declarations. Never edited by hand.

eslint.config.mjs ESLint flat config, extending `eslint-config-next`.

.prettierrc.json Prettier formatting rules, including the Tailwind class-sorting plugin.

.prettierignore Paths Prettier must skip (build output, lockfiles).

postcss.config.mjs PostCSS pipeline — loads the TailwindCSS v4 plugin.

components.json shadcn/ui generator config: 'new-york' style, neutral base colour, Lucide icons, and the `@/` components, `@/lib`, `@/hooks` import aliases.

vitest.config.ts Vitest setup for the unit and integration tests (environments, path aliases, setup files).

playwright.config.ts Playwright setup for end-to-end tests: base URL, the global seed step, per-role stored auth states, browser projects.

vercel.json Deployment config. Registers the daily Vercel Cron job that calls `/api/cron/sla-check` for SLA breach detection.

.env.example Documented template of every environment variable the app needs — copy to `.env.local` and fill in.

.env.local The real local secrets (Mongo URI, Auth.js secret, R2, Resend and Upstash keys). Git-ignored, never committed.

.gitignore Files excluded from version control: `node_modules`, `.next`, `.env.local`, Playwright auth states and artifacts.

.claude/ Claude Code workspace settings for this repository.

- | **settings.local.json** Locally approved tool and command permissions so routine commands run without re-prompting.

- | **launch.json** Launch configuration used when starting the app from the assistant.

.vscode/ Editor settings shared with the project.

- | **settings.json** Workspace-level VS Code preferences (formatter, format-on-save).

.next/ Next.js build output — compiled pages, server chunks and caches. Regenerated on every build or dev run; git-ignored.

node_modules/ Installed npm packages. Generated by `npm install`; git-ignored.

public/ Static files served directly at the site root — currently the default starter icons `file.svg`, `globe.svg` and `window.svg`.

2. docs/ — Project Documentation

The written specification the code is built against. These are the reference documents for the research paper as well as for development.

docs/ All written specifications, design references and testing records.

- | **architecture.md** The architecture blueprint written before implementation: the intended folder structure, database schema, authentication flow, RBAC flow, complaint lifecycle, routing and SLA design, reporting design and open questions.

- | **business-rules.md** The complete business-rule catalogue BR-001 to BR-100 (plus later additions such as BR-101, withdrawal). Every model, service and test in the codebase cites these IDs.

- | **business-rules.pdf** PDF rendering of the same business-rules catalogue, for submission alongside the paper.

- | **schema-reconciliation.md** Record of how the pre-written blueprint schema was reconciled with the business rules once BR-001 to BR-100 arrived — what changed in each collection, what was added (Assignment, GeneratedReport) and why.

- | **environment-variables.md** Reference for every environment variable: what it does, where to obtain the real value, and which dashboard or file it belongs in.

- | **data-flow-diagram.pdf** Data-flow diagram of the system, for the documentation chapter.
- | **functional-diagram.pdf** Functional decomposition diagram of the system's modules.
- | **swimlane-diagram.pdf** Swimlane diagram of the complaint-handling process across the student, staff, dean, QA and admin roles.
- | **uml-use-case-diagram.pdf** UML use-case diagram showing each actor and the use cases available to them.
- | **testing/** Test evidence and coverage mapping.
 - | **br-traceability.md** Traceability matrix mapping each business rule to the test that proves it, with a status marker per rule.
 - | **br-test-results.md** Recorded results of the full Vitest and Playwright runs, including which findings were found and fixed.
- documents-for-paper/** Exported diagrams kept separate for direct inclusion in the research paper.
 - | **ERD.png** Original entity-relationship diagram image.
 - | **ERD-reconciled.png** ERD updated to match the reconciled schema actually implemented in src/models.
 - | **ERD-reconciled.pdf** Print-quality PDF version of the reconciled ERD.

3. emails/ — Transactional Email Templates

React Email components rendered to HTML by the notification service and delivered through Resend. They sit outside src/ because they are a separate rendering target that can be previewed on its own.

emails/ All outbound email templates for the system.

- | **components/** Shared building blocks for the templates.
 - | **EmailLayout.tsx** Common wrapper: HTML shell, header, body container and footer, so every email looks consistent.
- | **ComplaintSubmitted.tsx** Confirmation sent to a student when their complaint is filed, carrying the generated ticket number.
- | **ComplaintAssigned.tsx** Notice to office staff that a complaint has been routed or assigned to them.
- | **ComplaintUpdated.tsx** Notice to the student when their complaint's status changes.
- | **ComplaintResolved.tsx** Notice that a complaint has been resolved, prompting the student to rate the service.
- | **Escalation.tsx** Alert sent when a complaint is escalated, for example to the dean, after a breached deadline.
- | **SLAWarning.tsx** Warning sent as an SLA response or resolution deadline approaches.
- | **PasswordReset.tsx** Password-reset email carrying the single-use reset link.

4. src/app/ — Routes (Next.js App Router)

Every URL in the application. Folders are URL segments; `page.tsx` renders a screen, `layout.tsx` wraps a role's screens, `loading.tsx` is the skeleton shown while server data loads, and `route.ts` under `api/` is an HTTP endpoint. A bracketed folder such as `[id]` is a dynamic parameter.

src/app/ Root of the routing tree.

- | **layout.tsx** Root layout: HTML shell, fonts, global providers and the toast container.
- | **globals.css** Global stylesheet — the Tailwind v4 import plus the design tokens used across the UI.
- | **page.tsx** Landing page at `/`, which sends visitors to login or to their role's dashboard.
- | **error.tsx** Global error boundary shown when a server render throws.
- | **not-found.tsx** The 404 page.
- | **favicon.ico** Browser tab icon.
- | **dashboard/page.tsx** Neutral `/dashboard` entry point that redirects each user to the dashboard for their role.
- | **(public)/** Route group for pages reachable without logging in. The parentheses group these routes without adding a URL segment.
 - | **layout.tsx** Centred, minimal layout used by all the authentication screens.
 - | **login/page.tsx** Sign-in form; on success the user is redirected to their role's home route.
 - | **register/page.tsx + register-form.tsx** Student self-registration page and its client-side form with inline validation.
 - | **forgot-password/page.tsx** Requests a password-reset link by email.
 - | **reset-password/[token]/page.tsx** Sets a new password using the token from the reset email.
- | **student/** Screens for the student role — filing and tracking their own complaints.
 - | **layout.tsx / error.tsx** Student shell with navigation, notification bell and sign-out, plus an error boundary scoped to this area.
 - | **dashboard/page.tsx** Personal summary: counts by status, recent activity and shortcuts.
 - | **complaints/page.tsx** List of the student's own complaints with search, sorting and status filters.
 - | **complaints/new/page.tsx** The complaint submission form: category, title, description, priority and file attachments.
 - | **complaints/[id]/page.tsx** One complaint in detail — timeline, attachments, the satisfaction rating form, and edit or withdraw while it is still unassigned.
 - | **feedback/new/page.tsx** Standalone suggestion form, optionally anonymous, unrelated to any specific complaint.
 - | **notifications/page.tsx** The student's in-app notification inbox.
 - | **profile/page.tsx** Profile details and password change.
- | **staff/** Screens for office staff — the people who actually work the complaints.
 - | **layout.tsx** Staff shell and navigation.
 - | **dashboard/page.tsx** Workload overview: assigned, in progress, due soon and overdue counts.
 - | **complaints/page.tsx** The office queue, scoped to the staff member's own office.
 - | **complaints/[id]/page.tsx** Working view of a complaint: change status, add internal notes, assign to self, view attachments.
 - | **sla/page.tsx** SLA view of the staff member's caseload — what is approaching or past its deadline.
 - | **notifications/page.tsx / profile/page.tsx** Staff notification inbox, profile and password change.
- | **dean/** Screens for a college dean — oversight of complaints belonging to their college.
 - | **layout.tsx** Dean shell and navigation.
 - | **dashboard/page.tsx** College-level summary of volume, status mix and SLA health.
 - | **complaints/page.tsx** All complaints within the dean's college.
 - | **complaints/[id]/page.tsx** Complaint detail with the dean's reassignment control.
 - | **analytics/page.tsx** Charts for the college: monthly trend, category breakdown and priority mix.
 - | **reports/page.tsx** Generate and download reports scoped to the college.
 - | **notifications/page.tsx / profile/page.tsx** Dean notification inbox, profile and password change.
- | **qa/** Screens for the Quality Assurance office — institution-wide monitoring.
 - | **layout.tsx** QA shell and navigation.
 - | **dashboard/page.tsx** Institution-wide key figures and charts.

- | **complaints/page.tsx** Read-only view of all complaints across every office.
- | **complaints/[id]/page.tsx** Full detail and timeline of any complaint, for review.
- | **sla-compliance/page.tsx** SLA compliance monitoring, including the per-office heat map.
- | **feedback/page.tsx** Review of the standalone feedback and suggestions submitted by students.
- | **reports/page.tsx** Institution-wide report generation and download history.
- | **notifications/page.tsx / profile/page.tsx** QA notification inbox, profile and password change.
- | **admin/** Screens for the system administrator — the configuration and master-data side of the system.
 - | **layout.tsx** Admin shell and navigation.
 - | **dashboard/page.tsx** System health overview: users, offices, complaint volume and recent audit activity.
 - | **users/page.tsx, users/[id]/page.tsx** User management — create accounts of any role, edit, activate or deactivate, force logout.
 - | **offices/page.tsx, offices/[id]/page.tsx** Manage colleges and service offices and their active status.
 - | **categories/page.tsx, categories/new/, categories/[id]/** Manage complaint categories, their default priority and whether they can be submitted against.
 - | **sla-rules/page.tsx** Configure response and resolution deadlines per category and priority.
 - | **complaints/page.tsx, complaints/[id]/page.tsx** Administrative view of every complaint, including archiving.
 - | **audit-logs/page.tsx, audit-logs/[id]/page.tsx** Browse the immutable audit trail and inspect a single entry.
 - | **notifications/page.tsx** Admin inbox plus the broadcast composer for announcements to a whole role.
 - | **settings/page.tsx** Runtime system settings — ticket prefix, upload limits, lockout thresholds, session length — editable without redeploying.
 - | **reports/page.tsx / profile/page.tsx** Report generation with full system scope, and the admin's own profile.

5. src/app/api/ — HTTP Endpoints

Route handlers the browser and external services call. Each `route.ts` exports the HTTP verbs it supports. Every handler re-checks the session and the caller's role before touching data — the middleware layer alone is never trusted.

src/app/api/ All server endpoints.

- | **auth/[...nextauth]/route.ts** Auth.js catch-all handler powering sign-in, sign-out and the session endpoints.
- | **auth/register/route.ts** Student self-registration endpoint.
- | **me/route.ts** Returns the signed-in user's profile, and updates profile details or password.
- | **complaints/route.ts** GET lists complaints filtered and scoped to the caller's role; POST files a new one, running routing, SLA calculation, ticket numbering, timeline writing and notifications.
- | **complaints/[id]/route.ts** GET one complaint; PATCH updates it — status changes validated against the lifecycle state machine, plus student edit and withdrawal.
- | **complaints/[id]/assign/route.ts** Assign or reassign a complaint to an office or staff member, writing an immutable assignment record.
- | **complaints/[id]/notes/route.ts** List and create internal staff notes, which are never visible to students.
- | **complaints/[id]/attachments/route.ts** Register uploaded file metadata against a complaint, and list its attachments.
- | **complaints/[id]/attachments/[attachmentId]/download/route.ts** Issues a short-lived signed download URL for one attachment after checking access.
- | **complaints/[id]/rate/route.ts** Records the student's satisfaction rating and comment on a resolved complaint.
- | **uploads/presign/route.ts** Returns a presigned Cloudflare R2 URL so the browser uploads directly to storage; validates file type and size first.
- | **categories/route.ts** Public list of active, submittable categories used by the submission form.
- | **colleges/route.ts** List of colleges, used by registration and by filters.
- | **offices/route.ts, offices/[id]/staff/route.ts** List offices, and list the staff of one office for the assignment dropdown.
- | **feedback/route.ts** Submit and list standalone feedback and suggestions.
- | **notifications/route.ts** Lists the caller's notifications with the unread count.
- | **notifications/[id]/read/route.ts, notifications/read-all/route.ts** Mark one notification, or all of them, as read.
- | **analytics/trends/route.ts** Aggregated chart data — monthly trend, category and priority breakdowns — scoped to the caller's role.
- | **reports/export/route.ts** Starts a report generation: builds the dataset, renders CSV, Excel or PDF, uploads it and records it.
- | **reports/[id]/route.ts, reports/[id]/download/route.ts** Status and metadata of one generated report, and the access-checked download of its file.
- | **cron/sla-check/route.ts** Scheduled job target. Scans open complaints for approaching or breached deadlines, escalates them and sends warnings. Called daily by Vercel Cron.
- | **admin/users/route.ts + [id]/route.ts** Administrator user CRUD, activation and forced logout.
- | **admin/offices/route.ts + [id]/route.ts** Office and college CRUD.
- | **admin/categories/route.ts + [id]/route.ts** Category CRUD and activation.
- | **admin/categories/category-activation.integration.test.ts** Integration test proving that deactivating a category blocks new submissions without harming existing complaints.
- | **admin/sla-rules/route.ts + [id]/route.ts** SLA rule CRUD.
- | **admin/routing-rules/route.ts + [id]/route.ts** Routing rule CRUD — which category goes to which office.
- | **admin/complaints/[id]/route.ts** Administrative complaint actions such as archiving.
- | **admin/audit-logs/route.ts** Paginated, filterable read of the audit trail.
- | **admin/settings/route.ts** Read and update the runtime settings singleton.
- | **admin/notifications/broadcast/route.ts** Sends one announcement to every user in a chosen role.
- | **dev/seed/route.ts** Development-only endpoint that wipes and reseeds the database with known accounts, offices, categories and roughly 300 complaints. Used by the E2E suite.
- | **dev/test-email/route.ts** Development-only endpoint for sending a test email to verify the Resend configuration.

6. src/components/ — Reusable UI

Presentational and interactive React components, grouped by the role or concern that uses them. Pages compose these; the components themselves hold no business rules.

src/components/ All shared UI building blocks.

- | **ui/** Low-level primitives generated by shadcn/ui.
 - | **select.tsx** The themed Radix-based select used by every dropdown in the app.
- | **shared/** Components used by more than one role.
 - | **StatCard.tsx** The dashboard metric tile — label, value, icon and optional trend.
 - | **StatusBadge.tsx** Coloured pill rendering a complaint's status consistently everywhere.
 - | **ComplaintListRow.tsx** One row in any complaint list — ticket, title, status, office and dates.
 - | **TimelineEvent.tsx** A single entry in a complaint's activity timeline.
 - | **NotesSection.tsx** Internal notes thread with its composer.
 - | **AttachmentGallery.tsx** Lists a complaint's attachments and handles secure downloads.
 - | **NotificationsPanel.tsx, NotificationIcon.tsx** The notification inbox list, and the bell with its unread badge.
 - | **ListSearchInput.tsx, ListSortSelect.tsx** Search box and sort dropdown that drive list pages through URL query parameters.
 - | **ProfileSettingsForm.tsx** Shared profile and password-change form used by all five roles.
 - | **ConfirmDialog.tsx** Confirmation prompt shown before destructive or irreversible actions.
 - | **Toast.tsx** Transient success and error messages.
 - | **Skeleton.tsx** Loading placeholders used by the loading.tsx files.
 - | **RelativeTime.tsx** Renders timestamps as '2 hours ago' on the client, avoiding server/client time mismatches.
 - | **CopyButton.tsx** Copies a value, such as a ticket number, to the clipboard.
- | **student/** Student-only components.
 - | **StudentNav.tsx** Student navigation bar.
 - | **EditWithdrawComplaint.tsx** Lets a student edit or withdraw their complaint while it is still unassigned.
 - | **RatingForm.tsx** One-to-five satisfaction rating with an optional comment.
 - | **FeedbackForm.tsx** Standalone suggestion form with an anonymous option.
- | **staff/** Office-staff components.
 - | **StaffNav.tsx** Staff navigation bar.
 - | **StatusUpdateForm.tsx** Status dropdown limited to the transitions legal from the current status.
 - | **AssignSelfButton.tsx** Lets a staff member take ownership of a complaint in their office.
- | **dean/** Dean components.
 - | **DeanNav.tsx, ReassignForm.tsx** Dean navigation, and the control that reassigns a complaint to another office or staff member.
- | **qa/** Quality Assurance components.
 - | **QaNav.tsx** QA navigation bar.
 - | **SlaHeatmap.tsx** Office-by-office SLA compliance heat map.
 - | **charts/QaBarList.tsx, QaDonutChart.tsx, QaTrendChart.tsx** The chart components used on the QA dashboard.
- | **analytics/** Charts shared by the dean and QA analytics screens.
 - | **TrendChart.tsx** Monthly complaint volume line chart, built with Recharts.
 - | **CategoryBarChart.tsx** Complaints per category bar chart.
 - | **PriorityPieChart.tsx** Priority distribution pie chart.
 - | **TruncatedAxisTick.tsx** Custom axis label that shortens long category names so charts stay readable.
- | **reports/** Reporting UI.
 - | **ReportsPanel.tsx, ReportStatusBadge.tsx** The generate-and-download panel shared by the dean, QA and admin report pages, and the badge showing each report's generation state.
- | **admin/** Administrator components — mostly the modal editors behind the admin tables.
 - | **AdminNav.tsx** Admin navigation bar.
 - | **Modal.tsx, FormField.tsx** The dialog shell and labelled input used by every admin editor.

- | **EditUserButton.tsx, EditOfficeButton.tsx, EditCategoryButton.tsx, EditSlaRuleButton.tsx, EditRoutingRuleButton.tsx** Open the modal editor for each kind of master-data record.
- | **UserStatusToggle.tsx, OfficeStatusToggle.tsx, CategoryStatusToggle.tsx** Activate or deactivate a record in place.
- | **ForceLogoutButton.tsx** Invalidates a user's sessions immediately by bumping their token version.
- | **ArchiveComplaintToggle.tsx** Archives or restores a complaint.
- | **BroadcastComposer.tsx** Composes and sends an announcement to a whole role.
- | **auth/logout-button.tsx** Sign-out control used in every role's navigation.
- | **complaints, forms/** Reserved folders from the original blueprint, currently holding only .gitkeep placeholders.

7. src/features/ — Domain Logic

The business layer, organised by domain rather than by technical type. Each feature owns its Zod schemas (input validation), its services (the actual rules) and its tests. Route handlers stay thin and call into here, which is why the same rule cannot drift between two screens.

src/features/ All domain modules.

- | **auth/** Authentication, registration and the password lifecycle.
 - | **schemas/login.schema.ts, register.schema.ts, password-reset.schema.ts** Zod validation for each auth form, including the shared password-strength policy.
 - | **services/auth.service.ts** Credential verification: unique-email lookup, the account-must-be-active check, last-login stamping, and the failed-attempt counter with temporary lockout.
 - | **services/password.service.ts** Bcrypt hashing and verification, centralised so every call site uses the same cost factor and passwords are never handled in plaintext.
 - | **services/register.service.ts** Student self-registration — it only ever creates student accounts, since staff, dean, QA and admin accounts are administrator-created.
 - | **services/password-reset.service.ts** Issues, hashes, expires and consumes single-use reset tokens, and sends the reset email.
 - | **services/session.service.ts** Defence-in-depth session check that re-reads the user from the database, so a deactivated account or forced logout takes effect immediately rather than when the token expires.
 - | **actions.ts, actions/logout.action.ts** The Server Actions the auth forms submit to.
 - | ***.test.ts / *.integration.test.ts** Tests for login, lockout, registration, password hashing and the reset flow.
- | **complaints/** The core domain — the complaint aggregate.
 - | **schemas/complaint.schema.ts, assign.schema.ts** Validation for creating and updating a complaint, and for assignment payloads.
 - | **services/status-transitions.service.ts** The lifecycle state machine: which status may follow which, so an invalid change is rejected on the server.
 - | **services/ticket-number.service.ts** Generates the human-readable ticket number from an atomic per-year counter, so concurrent submissions can never collide on the same number.
 - | **complaints.integration.test.ts, notes.integration.test.ts, rating.integration.test.ts** Integration tests for submission, role scoping, notes and ratings, run against an in-memory MongoDB.
 - | **repository, hooks/** Reserved folders from the blueprint's repository-pattern plan; currently placeholders.
- | **routing-engine/** Automatic assignment of a new complaint to the right office.
 - | **schemas/routing-rule.schema.ts** Validation for routing rules.
 - | **services/routing.service.ts** Strict rule matching: if no active rule matches, or its target office is inactive, submission fails loudly instead of silently falling back to a default.
- | **sla/** Service-level agreements — deadlines and breach detection.
 - | **schemas/sla-rule.schema.ts** Validation for SLA rules.
 - | **services/sla.service.ts** Resolves the applicable rule for a category and priority, computes the response and resolution deadlines, and identifies what is approaching or already breached.
- | **notifications/** In-app notifications and outbound email.
 - | **schemas/broadcast.schema.ts** Validation for the administrator broadcast.
 - | **services/notification.service.ts** Creates notification records for the right recipients on each lifecycle event.
 - | **services/email.service.ts** Renders the templates in emails/ and dispatches them through Resend.
- | **reports/** Report generation and export.

- | **schemas/report.schema.ts** Validation of report type, format and filters, with defaults.
- | **services/report-data.service.ts** Builds the report dataset with MongoDB aggregation, applying the office, college, date-range and status filters.
- | **services/report-file.service.ts** Renders that dataset as CSV, as Excel via ExcelJS, or as PDF via pdfkit.
- | **services/report-upload.service.ts** Stores the finished file in Cloudflare R2 and returns its key.
- | **analytics/services/analytics.service.ts** The aggregation pipelines behind the dashboards: monthly trends, category and priority breakdowns and resolution times, scoped by role.
- | **audit-log/services/audit-log.service.ts** Writes the insert-only audit-trail entry for every sensitive action.
- | **attachments/schemas/attachment.schema.ts** Validation for presign requests and attachment metadata — file name, MIME type and size.
- | **feedback/** Standalone feedback: its category list and its submission schema.
- | **settings/** The runtime settings singleton — its schema, and the service that reads and updates it.
- | **admin/schemas/** Validation for administrator-managed master data: users, offices and categories.

8. src/models/ — Database Schema

Mongoose models: one file per MongoDB collection, defining fields, enums, indexes and references. These files are pure shape and persistence; the rules that use them live in `src/features`. A shared plugin verifies that every reference actually points at an existing document.

src/models/ All Mongoose models.

- | **User.ts** Accounts for all five roles: identity, hashed password, role, office or college link, active flag, failed-attempt counter, lockout time, last login and token version.
- | **Office.ts** Unified hierarchy for colleges and service offices — one collection with a type field, so routing and SLA rules can target either kind uniformly.
- | **Category.ts** Complaint categories with their default priority, target office and whether they are still submittable.
- | **Complaint.ts** The aggregate root: ticket number, submitter, category, title, description, priority, status, assigned office and staff, SLA deadlines, resolution details, rating and archive flag.
- | **ComplaintTimeline.ts** Append-only activity log for each complaint, kept as its own collection so long-lived tickets never approach MongoDB's 16 MB document limit.
- | **ComplaintNote.ts** Internal staff notes — role-gated, never shown to students, and create-only.
- | **Assignment.ts** One immutable record per assignment or reassignment: who assigned it, to whom, when and why. Preserves the handover history that the complaint's current-owner fields alone cannot.
- | **Attachment.ts** File metadata only — name, MIME type, size, storage key and uploader. The bytes themselves live in Cloudflare R2.
- | **Feedback.ts** Standalone suggestions and feedback not tied to any complaint, optionally anonymous.
- | **Notification.ts** In-app notification records with type, recipient, link and read state.
- | **AuditLog.ts** Immutable, insert-only trail of every sensitive action: actor, action, target, before and after values, IP address and timestamp.
- | **RoutingRule.ts** Maps a category, and optionally a priority condition, to the office that should receive it.
- | **SLARule.ts** Response and resolution deadlines per category and priority, with escalation settings.
- | **GeneratedReport.ts** A persisted record of each generated report: creator, type, format, filters, status and download location.
- | **PasswordResetToken.ts** Hashed, single-use, expiring password-reset tokens.
- | **Counter.ts** Generic atomic counter used for ticket-number sequencing through an upsert with `$inc`, which is race-free unlike a read-then-write approach.
- | **Settings.ts** Singleton document holding the admin-editable runtime configuration — ticket prefix, upload limits, lockout thresholds and session length.
- | **index.ts** Barrel file re-exporting every model for convenient imports.
- | ***.integration.test.ts** Model-level tests covering assignments, the timeline, generated reports, offices and cross-collection referential integrity.

9. src/lib/, src/middleware/ and the remaining src/ files

Cross-cutting infrastructure: the things every layer needs but that belong to no single domain.

src/lib/ Shared infrastructure and helpers.

- | **env.ts** The single place environment variables are read. A Zod schema validates every value at import time, so a missing or malformed setting fails immediately with a clear message instead of a mystery crash later.
- | **db.ts** The Mongoose connection singleton, cached on the global object so serverless invocations and dev hot-reloads reuse one connection rather than exhausting the Atlas connection limit.
- | **auth.config.ts** The Edge-safe half of the Auth.js configuration — session shape, callbacks and redirect pages, with no Node-only imports, so the middleware layer can load it.
- | **auth.ts** The full Auth.js configuration for the Node runtime, adding the Credentials provider that needs Mongoose and bcrypt.
- | **api-guards.ts** Reusable session and role guards that route handlers call first, returning 401 or 403 before any work is done.
- | **constants.ts** Central enum lists — roles, office types, priorities, complaint statuses, timeline event types, notification types and report formats — so a status name can never drift between files.
- | **status-transitions-client.ts** A client-safe mirror of the lifecycle map, used only to populate the staff status dropdown; the server revalidates every transition regardless.
- | **password.ts + password.test.ts** Bcrypt hash and verify helpers, with tests proving a password is never stored or compared in plaintext.
- | **r2.ts** The Cloudflare R2 (S3-compatible) client and a check that storage is actually configured.
- | **resend.ts** The Resend email client and its configuration check.
- | **rate-limit.ts** Upstash Redis rate limiting, applied to login, registration and other abusable endpoints.
- | **logger.ts** A dependency-free structured logger producing consistent, greppable log lines on any host.
- | **mongoose-ref-integrity.ts** Schema plugin that validates every ObjectId reference actually points at an existing document — a check Mongoose does not perform on its own.
- | **utils.ts** Small helpers: the `cn()` Tailwind class merger, and a regex escaper that makes user input safe inside a MongoDB search filter.

src/middleware/ The route access table used by the request interceptor.

- | **rbac.ts** Maps route prefixes to the role allowed on them and names each role's home route. Deliberately pure and dependency-free so it stays Edge-safe and independently testable.
- | **rbac.test.ts** Unit tests for every route and role combination in that table.

src/proxy.ts The Next.js 16 request interceptor. It runs before any route resolves, reads the session token, and redirects unauthenticated visitors to login and wrong-role users to their own home — the first, coarse layer of access control.

src/test/ Test scaffolding shared by the Vitest suites.

- | **setup-db.ts** Starts and tears down an in-memory MongoDB instance, so integration tests run against a real database engine without touching Atlas.
- | **fixtures.ts** Factory helpers that create test offices, users, categories and rules.

src/types/index.ts Shared TypeScript types and interfaces, populated as each domain is built.

src/hooks/ Reserved folder for shared React hooks; currently a placeholder.

10. tests/ — End-to-End Test Suite

Playwright specifications that drive a real browser against the running application, proving the business rules hold through the actual interface and not only at the service layer.

tests/e2e/ All Playwright specifications and their scaffolding.

- | **global-setup.ts** Runs once before the suite: calls the dev seed endpoint so every spec starts from a known data set — fixed accounts, colleges and offices, 13 categories, roughly 300 complaints and 24 feedback rows.
- | **auth.setup.ts** Logs in once per role and saves the session, so specs start already authenticated instead of repeating the login flow.
- | **fixtures.ts** Role-scoped test and expect wrappers that inject the right stored session.
- | **helpers.ts** Shared helpers, including the seeded test password and a one-off login as some other account.
- | **.auth/** Stored session files per role, generated by `auth.setup.ts` and git-ignored.

- | **auth.spec.ts** Login, logout, invalid credentials, lockout and deactivated-account behaviour.
- | **rbac-redirects.spec.ts** Confirms each role is blocked from every other role's area.
- | **session-revocation.spec.ts** Proves a forced logout or deactivation invalidates an active session immediately.
- | **complaint-submission.spec.ts** End-to-end submission, including ticket generation and automatic routing.
- | **complaint-status-lifecycle.spec.ts** Walks a complaint through its legal statuses and checks the illegal ones are rejected.
- | **edit-withdraw-complaint.spec.ts** Verifies a student may edit or withdraw only while the complaint is still unassigned.
- | **assign-scoping.spec.ts** Verifies staff and deans only ever see and act on complaints within their own scope.
- | **attachments.spec.ts** Upload, listing and access-controlled download of attachments.
- | **notes-and-notifications.spec.ts** Internal notes staying hidden from students, and notifications reaching the right recipients.
- | **rating.spec.ts** Satisfaction rating being available only after resolution, and only once.
- | **analytics.spec.ts** Dashboard and chart rendering with correctly scoped data.
- | **reports.spec.ts** Report generation and download in each format.
- | **admin-crud.spec.ts** Administrator management of users, offices, categories and rules.

tests/unit, tests/integration/ Placeholder folders from the original layout. The unit and integration tests actually live beside the code they cover, inside `src/`.